

## 补丁应用 (patch)

### 【题目背景】

西西艾弗岛运营公司的信息技术部门，需要协作开展程序开发和代码审查工作。他们的工作流程是这样的：首先，开发者将代码复制一份，并修改代码副本，从而得到期望的代码。然后，开发者使用 `diff` 工具比较修改前后的代码的区别，并将其输出用邮件发送给代码审查者。审查者收到邮件后，可以直接观察开发者作出的代码变更，并提出意见。反复进行后，当代码审查者对变更满意时，会使用 `patch` 程序，将开发者者提出的修改应用到原代码上，从而得到最终的代码。

现在，他们已经可以实现 `diff` 程序，但是需要你帮助他们实现这个 `patch` 程序。

`diff` 的输出称作补丁。补丁由一个或多个块组成。每个块包含若干行文本，表示对文件的一处修改。其中第一行以 `@@` 开头和结尾，形如：

```
1 @@ -NN,MM +nn,mm @@
```

其中 `NN`、`MM`、`nn`、`mm` 表示一个 1 至 9 之间的字符和零个或多个 0 至 9 之间的字符组成的字符串，表示一个正整数。每块的第一行表示原文件和新文件的行号范围。其中，`NN` 表示这处修改在原文件的第 `NN` 行开始（原文件的行号从 1 开始编号）；`MM` 表示这处修改涉及原文件的 `MM` 行；`nn` 表示这处修改，在修改后从新文件的第 `nn` 行开始；`mm` 表示这处修改在修改后，在新文件中有 `mm` 行。

随后会有若干行文本，表示修改的内容。如果一行文本以 `-` 开头，表示这行文本在原文件中被删除；如果一行文本以 `+` 开头，表示这行文本在新文件中被添加；如果一行文本以空格开头，表示这行文本在原文件和新文件中都存在，未发生变化。因此，一个块中所有以 `-` 开头的行和以空格开头的行的总数，应该等于 `MM`；一个块中所有以 `+` 开头的行和以空格开头的行的总数，应该等于 `mm`。一处修改的描述中，可以适当包含若干不变的行，以便确定修改的上下文。

例如，下面是一个块的内容：

```
1 @@ -1,4 +1,5 @@
2 -a
3 +b
4 1
5 +c
6 2
7 3
```

表示该处修改自原文件的第 1 行开始，共 4 行；修改后的文本从新文件的第 1 行开始，共 5 行。此处修改前，原文件的内容应该为：

```
1 a
```

```
2 1
3 2
4 3
```

修改后，新文件的内容应该为：

```
1 b
2 1
3 c
4 2
5 3
```

### 【题目描述】

但是，`patch` 程序在处理 `diff` 的输出时，对格式的要求可以较为宽松。例如，它可以允许块内有注释，也可以允许块的行号与实际原文件的行号不匹配。这是因为，在实际应用中，`diff` 生成后，源文件可能经历了其它的变更，导致行号出现了挪动。`patch` 程序的具体工作过程是：

1. 读取全部输入，将 `#` 开头的行视为注释，并移除；
2. 寻找 `@` 开头的行，并将该行至下一个 `@` 开头的行（或文本结尾）之间的内容视为一个块，如果没有找到 `@` 开头的行，则认为补丁损坏；
3. 从前到后依次对每个块：
  1. 解析第一行，检查其格式是否正确，如果不正确，则认为补丁损坏；
  2. 解析出 `NN`、`MM`、`nn`、`mm`，其中忽略 `nn`；
  3. 如果这个块不是第一个块，检查 `NN` 是否不小于前一个块的 `NN` 与 `MM` 之和，如果不是，则认为补丁损坏；
  4. 解析其余行，如果这些行中存在不是以 `-`、`+`、空格开头的行，则认为补丁损坏；
  5. 将块中所有以 `-` 开头的行和以空格开头的行提取出来，作为原文件的内容片段；
  6. 检查原文件的内容片段的行数是否与 `MM` 一致，如果不一致，则认为补丁损坏；
  7. 将块中所有以 `+` 开头的行和以空格开头的行提取出来，作为新文件的内容片段；
  8. 检查新文件的内容片段的行数是否与 `mm` 一致，如果不一致，则认为补丁损坏；
4. 如果所有块都通过了检查，则对于每个块：

1. 检查是否存在绝对值小于  $MM$  的整数  $\delta$ ，使得自原文件的第  $NN + \delta$  行开始的  $MM$  行，与块的原文件内容片段完全匹配。如果不是第一个块，还需满足  $NN + \delta$  不小于前一个块的  $NN$  与  $MM$  之和，即满足每块对应的原文区域没有重叠。如果不存在这样的  $\delta$ ，则认为补丁损坏；
2. 如果存在多个这样的  $\delta$ ，则取绝对值最小的那个，如果仍然存在多个，则取最小的那个；
3. 将原文件的第  $NN + \delta$  行开始的  $MM$  行替换为块的新文件内容片段；
4. 将该块和此后的所有块的  $NN$  加上  $\delta$ 。

### 【输入格式】

从标准输入读入数据。

输入的第一行包含一个正整数  $n$ ，表示原文件的总行数。

接下来的  $n$  行文本，表示原文件的内容。

接下来的若干行，表示待应用的补丁，其中补丁可能是损坏的。

### 【输出格式】

输出到标准输出。

输出应用补丁后的文件内容；如果补丁损坏，输出 `Patch is damaged..`

### 【样例 1 输入】

```
1 7
2 bbb
3 a
4 1
5 2
6 3
7 4
8 5
9 dummy
10 @@ -1,4 +1,5 @@
11 -a
12 +b
13 1
14 +c
15 2
16 3
```

```
17 @@ -6,2 +6,2 @@
18 -4
19 +6
20 5
```

**【样例 1 输出】**

```
1 bbb
2 b
3 1
4 c
5 2
6 3
7 6
8 5
```

**【样例 1 解释】**

输入原始文本共有 7 行，之后开始解析补丁。遇到的第一个以 @ 开头的行是 @@ -1,4 +1,5 @@，表示第一个块内容的开始，因此忽略此前的 dummy 行，即输入的第 8 行。第一个块即为题目描述中的块，该块的原始内容与输入的原始内容的第 2 行到第 5 行相同，因此对应将这些行替换，且将该块及其后的所有块的行号加上 1。所以此时输出 bbb（未包括于第一个块）、b、1、c、2、3。第二个块的头部标识该块起始于原始文件的第 6 行，加上 1 为第 7 行，该块的原始内容共两行，分别为 4、5。这一内容与原始文件的第 6 行、第 7 行相同，因此对应将这两行替换，且将该块及其后的所有块的行号减去 1。所以此时接下来输出 6、5。

**【样例 2 输入】**

```
1 7
2 bbb
3 a
4 1
5 2
6 3
7 4
8 5
```



```
9 dummy
```

**【样例 2 输出】**

```
1 Patch is damaged.
```

**【样例 2 解释】**

输入原始文本共有 7 行，之后开始解析补丁。补丁中不含有有效的块，因此为非法补丁。

**【样例 3 输入】**

```
1 8
2 bbb
3 a
4 1
5 2
6 3
7 4
8 5
9 6
10 dummy
11 @@ -1,4 +1,5 @@
12 -a
13 +b
14 1
15 +c
16 2
17 3
18 @@ -6,2 +6,2 @@
19 -4
20 +6
21 5
22 6
```

**【样例 3 输出】**

```
1 Patch is damaged.
```

**【样例 3 解释】**

输入原始文本共有 8 行，之后开始解析补丁。补丁的第二个块中，其第一行表示其原始内容有 2 行，但随后却给出了 3 行的原始内容，因此为非法补丁。

**【样例 4 输入】**

```
1 8
2 bbb
3 a
4 1
5 2
6 3
7 4
8 5
9 6
10 dummy
11 @@ -1,4 +1,5 @@
12 -a
13 +b
14 1
15 +c
16 2
17 3
18 @@ -6,2 +6,2 @@
19 -4
20 +6
21 5
22 # 6
```

**【样例 4 输出】**

```
1 bbb
```

```
2 b
3 1
4 c
5 2
6 3
7 6
8 5
9 6
```

#### 【样例 4 解释】

与样例 3 相比，多出来的一行前增加了 # 号被忽略，因此为合法补丁，其应用过程与样例 1 类似。

#### 【子任务】

对于 30% 的数据，补丁仅包含一个块，且不含有注释。

对于 60% 的数据，有  $n \leq 60$ ，且输入中行的长度不超过 120，补丁是合法补丁，补丁块相对于原始内容无偏移。

对于 100% 的数据，有  $n \leq 2000$ ，且输入中行的长度不超过 830，且输入的总长度不超过 500KiB，输入数据中仅包含 ASCII 码在 32 至 126 之间的字符和换行符（ASCII 码为 10），且补丁块不超过 25 个。

#### 【评分方式】

本题共包括十个测试点：

- 前六个测试点每通过一个可得 10 分；
- 后四个测试点同时通过可得 40 分，否则不得分。换言之，对所有测试点皆输出补丁损坏不会得分。

#### 【提示】

本题目的输入数据中，每一行，包括最后一行在内，都以换行符（ASCII 码为 10，即 16 进制的 0x0a，亦为 escape 序列 \n）结束。